小型数据库系统设计与实现实验报告

一、实验目的

本实验旨在通过设计并实现一个简化版的数据库系统，加深对编译原理中词法分析、语法分析、语义执行等内容的理解；同时掌握 SQL 基本语句的解析与执行机制，理解数据库管理系统（DBMS）的基本结构与工作原理。

二、实验环境

- **操作系统**：Windows 10
- **开发语言**：Python 3.10
- **依赖工具**：
  - ply：Python Lex-Yacc，实现词法和语法分析
  - uv：轻量级包管理器，用于构建虚拟环境

三、系统结构设计

模块	功能描述
lexer.py	实现 SQL 的词法分析，识别关键词、标识符、字符串、数字、运算符等
parser.py	实现 SQL 的语法分析与语义动作，构建抽象语法树，并将执行任务传给执行器
executor.py	执行语句的具体逻辑，包括数据插入、查询、更新、删除等操作
catalog.py	管理元数据，如数据库、表结构信息等
shell.py	提供交互式命令行，模拟数据库控制台

3.1.词法分析

在 lexer.py 中，我们使用 PLY 的 lex 模块实现词法分析器：

- 首先定义保留字字典 reserved，其中包含 SQL 中的关键字，如 select、from、where、insert 等。
- 然后定义 token 列表 tokens，除了保留字外，还包括：
  - **标识符（ID）**
  - **字符串常量（STRING）**
  - **整型常量（NUMBER）**
  - **运算符（EQ、NE、GT、LT、GE、LE）**
  - **分隔符（COMMA, SEMICOLON, LPAREN, RPAREN 等）**

对于简单 token（如 +, =, ;），通过正则表达式直接定义；
对于复杂 token（如 ID, NUMBER, STRING），使用函数处理，例如：

3.2.语法分析

在 parser.py 中，我们使用 PLY 的 yacc 模块定义语法规则，支持如下 SQL 语句：

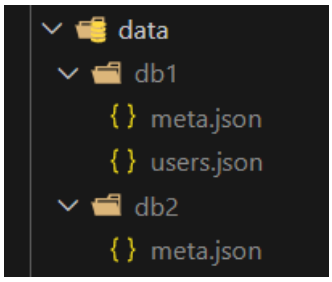
- CREATE DATABASE / TABLE
- USE DATABASE
- INSERT INTO
- SELECT [字段] FROM 表名 [WHERE 表达式]
- UPDATE 表名 SET 字段=值 [WHERE 表达式]
- DELETE FROM 表名 [WHERE 表达式]
- DROP TABLE / DATABASE
- SHOW TABLES

其中表达式部分（用于 WHERE 子句）支持括号嵌套、布尔运算（AND / OR）、比较运算等，并通过优先级声明解决语法冲突：

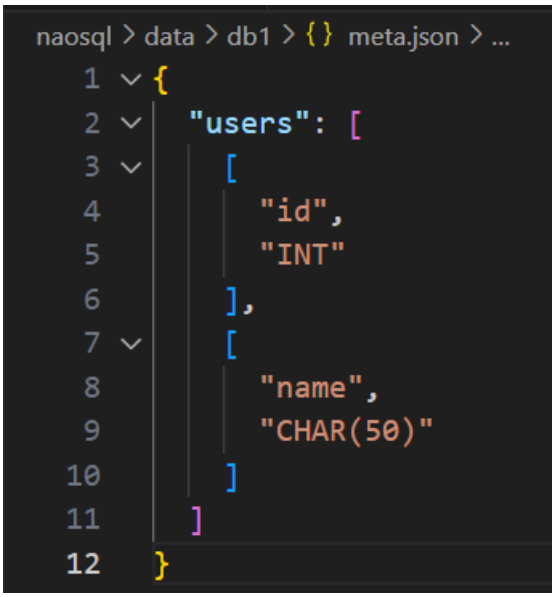
```
# 定义优先级
precedence = (
    ("left", "OR"),
    ("left", "AND"),
    ("nonassoc", "EQ", "NE", "LT", "LE", "GT", "GE"),
    )
```

3.3.存储的物理结构

数据库采用**文件存储**方式：每个数据库对应一个文件夹，元数据保存在 meta.json，表数据保存在 {table}.json 中。



如图，创建了数据库 db1 和 db2，它们都有 meta.json 文件来存储表结构元数据。db1 中有一个数据表 users.json，其表结构在 meta.json 中存储，如下图：



users 表的数据如下图：

naosql > data > db1 > { } users.json > ...

```
[
  {
    "id": 1,
    "name": "Alice"
  },
  {
    "id": 2,
    "name": "Bob"
  },
  {
    "id": 3,
    "name": "Peter"
  },
  {
    "id": 4,
    "name": "Eve"
  }
]
```

存储表的整体是一个列表，每一条记录是一个 json 对象。

四、支持的 SQL 子集

本系统支持如下简化 SQL 指令：

数据库操作

- CREATE DATABASE dbname;
- USE dbname;
- DROP DATABASE dbname;
- SHOW TABLES;

表操作

- CREATE TABLE table_name (col1 TYPE, col2 TYPE, ...);
- DROP TABLE table_name;

数据操作

- INSERT INTO table_name VALUES (val1, val2, ...);
- SELECT * FROM table_name [WHERE 条件];
- SELECT col1, col2 FROM table_name [WHERE 条件];
- UPDATE table_name SET col = val [WHERE 条件];
- DELETE FROM table_name [WHERE 条件];

支持的数据类型包括：

- INT
- CHAR(n)

支持的 WHERE 表达式包括：

- 比较运算：=, !=, <, <=, >, >=
- 逻辑运算：AND, OR
- 括号嵌套：(...)

五、功能展示与测试

```
naoSQL> CREATE DATABASE db1;
数据库 db1 创建成功
naoSQL> USE db1;
已切换至数据库 db1
naoSQL(data\db1)> CREATE TABLE users (
>     id INT,
>     name CHAR(50)
> );
表 users 创建成功
naoSQL(data\db1)> INSERT INTO users VALUES (1, 'Alice');
插入成功：{'id': 1, 'name': 'Alice'}
naoSQL(data\db1)> INSERT INTO users VALUES (2, 'Bob');
插入成功：{'id': 2, 'name': 'Bob'}
naoSQL(data\db1)> INSERT INTO users VALUES (3, 'Peter');
插入成功：{'id': 3, 'name': 'Peter'}
naoSQL(data\db1)> INSERT INTO users VALUES (4, 'Eve');
插入成功：{'id': 4, 'name': 'Eve'}
naoSQL(data\db1)> SELECT * FROM users;
{'id': 1, 'name': 'Alice'}
{'id': 2, 'name': 'Bob'}
{'id': 3, 'name': 'Peter'}
{'id': 4, 'name': 'Eve'}
naoSQL(data\db1)> SELECT name FROM users WHERE name = 'Alice';
{'name': 'Alice'}
naoSQL(data\db1)> SELECT id, name FROM users WHERE id <= 2 OR name = 'Eve';
{'id': 1, 'name': 'Alice'}
{'id': 2, 'name': 'Bob'}
{'id': 4, 'name': 'Eve'}
naoSQL(data\db1)> UPDATE users SET name = 'Charlie' WHERE id = 1;
更新了 1 条记录
naoSQL(data\db1)> DELETE FROM users WHERE id = 4;
删除了 1 条记录
naoSQL(data\db1)> SELECT * FROM users;
{'id': 1, 'name': 'Charlie'}
{'id': 2, 'name': 'Bob'}
{'id': 3, 'name': 'Peter'}
naoSQL(data\db1)> DROP TABLE users;
表 users 删除成功
naoSQL(data\db1)> DROP DATABASE db1;
数据库 db1 删除成功
naoSQL>
```

六、核心技术难点与解决方案

1. 表达式解析冲突

- 在 WHERE 子句中使用复杂逻辑表达式（如 a = 1 OR b != 2 AND ...）时，容易出现语法冲突。
- 解决方案：通过 precedence 声明指定 AND, OR, 比较运算的优先级和结合性，成功消除了 shift/reduce 冲突。

2. 字段值解析歧义

- 诸如 id = 1 中的 id 必须识别为列名而非字符串常量。
- 解决方案：在 value 规则中支持 ID 类型，并在执行器中判断字段名是否存在于当前行。

七、项目不足与缺点

尽管本系统已实现了基本的 SQL 数据库功能，但由于时间与精力所限，仍存在以下不足之处：

1. 数据类型支持有限

目前仅支持 INT 和 CHAR(n) 两种基础数据类型，缺乏如 FLOAT、DATE、BOOLEAN 等常见类型，也不支持类型检查、自动类型转换和默认值设置，限制了实际数据建模的灵活性。

2. 无法处理复杂的 SQL 查询

系统仅支持基本的 SELECT-FROM-WHERE 查询语句，不支持：

- 多表连接（JOIN）
- 子查询（SELECT 内嵌 SELECT）
- 聚合函数（如 COUNT, SUM, AVG）
- 分组与排序（GROUP BY, ORDER BY）

这导致查询能力相对受限，无法覆盖真实数据库中的复杂分析任务。

3. 元数据与数据分离薄弱

虽然系统通过 meta.json 和 table.json 分别存储元数据与数据，但未引入事务机制、并发控制、缓存优化等机制，**在大规模并发访问时可能出现数据一致性问题或读写冲突**。

4. **不支持索引**

没有实现索引，也因此不能按索引高效查询。

5. **表结构与数据未做完整性约束检查**

未支持如 **主键、外键、非空约束、唯一性** 等数据库基本完整性约束，因此可以插入逻辑上冲突的数据记录。例如：

INSERT INTO users VALUES (1, NULL); -- 无报错

这种松散的数据校验机制，降低了数据可靠性。

6. **错误处理不完善**

- 错误提示不够详细、格式不统一；
- 某些异常情况未捕获，可能导致崩溃或产生错误结果；
- 错误行号与位置提示不精确，调试成本较高。

七、实验收获与体会

本次实验使我深入理解了从词法识别到语义执行的一整套语言处理流程。在构建数据库语言时，我不仅体会到了编译器前端的构造思维，还在执行器中体验了数据库系统中元数据管理、数据操作、表达式求值等后端机制的实际落地。通过亲手实现 SELECT、UPDATE 等语句的逻辑解析与执行，我对 SQL 的本质、解析器冲突的本源、以及表达式语义树的构造方法都有了更加直观的认知。